

École nationale supérieure en systèmes avancés et réseaux

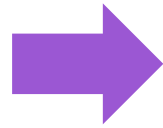
# Outils de développement<sup>1</sup>

## Partie I : Java, Javadoc et JUnit

Stéphanie CHOLLET

<sup>1</sup> *Software development tools that you can use to make your life easier*  
[J. Ferguson Smart, 2008]

- Le cycle de vie logiciel est un processus qui permet de produire un logiciel qui fonctionne



Pas seulement coder !

Normalisation du code

Gestion de la documentation

Scripts pour la compilation,  
le packaging et le déploiement  
d'applications de manière reproductible

Gestion des versions  
*Version Control System*

Plate-forme de tests

Suivi des problèmes  
*Bug tracker*

Analyse statique  
du code

Et ...

# Bien coder en Java

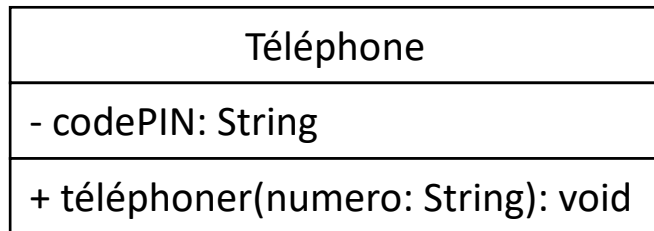
- Langage de **programmation de haut niveau orienté objet**, créé par James Gosling et Patrick Naughton, employés de Sun Microsystems, présenté officiellement le 23 mai 1995.
  - La programmation orientée objet est un paradigme de programmation informatique. Elle permet la définition et l'interactions de briques logicielles appelées objets. Un objet représente un concept, une idée ou toute entité du monde physique.

[https://fr.wikipedia.org/wiki/Java\\_\(langage\)](https://fr.wikipedia.org/wiki/Java_(langage))

[https://fr.wikipedia.org/wiki/Programmation orientée objet](https://fr.wikipedia.org/wiki/Programmation_orient%C3%A9e_objet)

# Objets et classes – 1/2

- Une **classe** est un modèle pour créer des objets qui ont des caractéristiques communes. La classe comporte la **structure d'un objet** (ses attributs et ses méthodes).
- En UML :



*Diagramme de classes*

```

public class Telephone {

    private String codePIN;

    public Telephone() {
        super();
    }

    public Telephone(String codePIN) {
        super();
        this.codePIN = codePIN;
    }

    public void téléphoner(String numero) {
        //Code pour téléphoner
    }

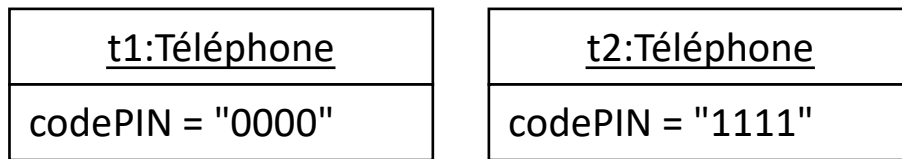
    public String getCodePIN() {
        return codePIN;
    }

    public void setCodePIN(String codePIN) {
        this.codePIN = codePIN;
    }

}
  
```

## Objets et classes – 2/2

- Un **objet** possède des données (**attributs**) et des comportements (**méthodes**). Les objets appartiennent à une classe qui définit le type.
- En UML :



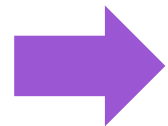
*Diagramme d'objets*

```

Telephone t1 = new Telephone("0000");
System.out.println(t1.getCodePIN());
t1.telephoner("0475750000");
  
```

```

Telephone t2 = new Telephone();
t2.setCodePIN("1111");
System.out.println(t2.getCodePIN());
  
```



Un objet est une instance d'une classe

# La ou les constructeurs

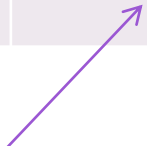
- Le rôle du constructeur est de permettre d'initialiser les données membre de la classe, ainsi que de permettre différentes actions (définies par le concepteur de la classe) lors de l'instanciation.
- Un constructeur :
  - Porte le même nom que la classe dans laquelle il est défini
  - N'a pas de type de retour (même pas *void*)
  - Peut avoir des arguments
  - Sa définition n'est pas obligatoire lorsqu'il n'est pas nécessaire :

```
public Telephone() {  
    super();  
}  
  
public Telephone(String codePIN) {  
    super();  
    this.codePIN = codePIN;  
}
```

- Pour les attributs et les méthodes :

| Visibilité | En UML | En Java   | Explications                                                                                                                    |
|------------|--------|-----------|---------------------------------------------------------------------------------------------------------------------------------|
| Publique   | +      | public    | Accessible partout où la classe est accessible, hérité par les sous-classes                                                     |
| Privée     | -      | private   | Accessible que depuis la classe elle-même                                                                                       |
| Protégée   | #      | protected | Accessible depuis la classe elle-même et hérité par les sous-classes, accessible aussi par le code situé dans le même paquetage |
| Paquetage  | ~      |           | Accessible que par le code situé dans le même paquetage et hérité que par les sous-classes du même paquetage                    |

Rien



## Visibilités en Java – 2/2

|           | Classe | Package | Sous-classe | Reste du monde |
|-----------|--------|---------|-------------|----------------|
| private   | Oui    | Non     | Non         | Non            |
| paquetage | Oui    | Oui     | Non         | Non            |
| protected | Oui    | Oui     | Oui         | Non            |
| public    | Oui    | Oui     | Oui         | Oui            |

```

public class A {
    private String a1;
    String a2;
    protected String a3;
    public String a4;
}
  
```

- ➔ Structurer correctement le code avec des paquetages
- ➔ Mauvaise spécification → ouverture de failles de sécurité
- ➔ Privilégier les attributs privés → accesseurs pertinents

# Convention de nommage – 1/2

- Exemple : <https://www.oracle.com/technetwork/java/codeconventions-135099.html>

| Identifier Type | Rules for Naming                                                                                                                                                                                                                                                                                                              | Examples                                                                                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Packages        | <p>The prefix of a unique package name is <b>always written in all-lowercase ASCII letters</b> and should be one of the top-level domain names. Subsequent components of the package name vary according to an organization's own internal naming convention.</p> <p><b><math>^[a-z_]+(\.[a-z_][a-z0-9_]*)*\\$</math></b></p> | <p>com.sun.eng<br/>com.apple.quicktime.v2<br/>edu.cmu.cs.bovik.cheese</p> <p>fr.esisar.in420</p> |
| Classes         | <p>Class names should be <b>nouns</b>, in mixed case with <b>the first letter</b> or each internal word <b>capitalized</b>.</p> <p><b><math>^[A-Z][a-zA-Z0-9]*\\$</math></b></p>                                                                                                                                              | <p>class Raster<br/>class ImageSprite</p>                                                        |
| Interfaces      | <p>Interface names should be <b>capitalized like class names</b>.</p> <p><b><math>^[A-Z][a-zA-Z0-9]*\\$</math></b></p>                                                                                                                                                                                                        | <p>interface RasterDelegate<br/>interface Storing</p>                                            |

## Convention de nommage – 2/2

| Identifier Type | Rules for Naming                                                                                                                                                                                                                 | Examples                                                                                                       |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Methods         | <p>Methods should be <b>verbs</b>, in mixed case with the <b>first letter lowercase</b>, with the first letter of each internal word capitalized.</p> <p><math>^{\wedge}[a-z][a-zA-Z0-9]^{\ast}\\$</math></p>                    | <pre>run(); runFast(); getBackground();</pre>                                                                  |
| Variables       | <p>Except for variables, all instance, class, and class constants are in mixed case with a <b>lowercase first letter</b>. Internal words start with capital letters.</p> <p><math>^{\wedge}[a-z][a-zA-Z0-9]^{\ast}\\$</math></p> | <pre>int i; char c; float myWidth;</pre>                                                                       |
| Constants       | <p>The names of variables declared class constants and of ANSI constants should be <b>all uppercase with words separated by undercores</b>.</p> <p><math>^{\wedge}[A-Z][A-Z0-9]^{\ast}(\_[A-Z0-9]^{\ast})^{\ast}\\$</math></p>   | <pre>static final int MIN_WIDTH = 4; static final int MAX_WIDTH = 999; static final int GET_THE_CPU = 1;</pre> |

# Quelques mots clés particuliers...

- static
  - Définition unique quel que soit le nombre d'objets instanciés
  - Permet de partager une variable de classe entre toutes les instances d'une même classe

```
public class Telephone {

    private static int compteur = 0;

    ...

}
```

| Téléphone                   |
|-----------------------------|
| - <u>compteur</u> : int = 0 |
|                             |

```
Telephone t1 = new Telephone("0000");
Telephone t2 = new Telephone("1111");

System.out.println(t1.getCompteur());      0
System.out.println(t2.getCompteur());      0
t1.setCompteur(4);
System.out.println(t1.getCompteur());      4
System.out.println(t2.getCompteur());      4
```

# Quelques mots clés particuliers...

- final
  - Permet de rendre l'entité sur laquelle il s'applique non modifiable une fois qu'elle a été déclarée et initialisée

```
public class Telephone {
    public final int nbTouches = 12;
    ...
}
```

| Téléphone                       |
|---------------------------------|
| - compteur: int = 12 {readOnly} |
|                                 |

ou

```
public class Telephone {
    public final int nbTouches;

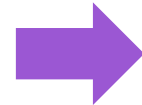
    public Telephone(String nbTouches) {
        super();
        this.nbTouches = nbTouches;
    }
}
```

## Quelques mots clés particuliers...

- Définition d'une constante :

```
public static final int NB_TOUCHES = 12;
```

| Téléphone                          |
|------------------------------------|
| - NB TOUCHES : int = 12 {readOnly} |
|                                    |



Attention à l'ordre des termes !

# Quelques méthodes particulières...

- Exécution du corps de l'application

```
public static void main(String[] args) {  
    for(int i = 0 ; i < args.length ; i++) {  
        System.out.print(args[i] + " ");  
    }  
    System.out.println();  
}
```

- La méthode toString()

```
@Override  
public String toString() {  
    return "Telephone [codePIN=" + codePIN + "];"  
}
```

- Les accesseurs et mutateurs : getter() et setter()

```
public String getCodePIN() {  
    return codePIN;  
}  
  
public void setCodePIN(String codePIN) {  
    this.codePIN = codePIN;  
}
```

# Quelques méthodes particulières...

- equals()
  - Vérifie si 2 instances sont sémantiquement équivalentes même si ce sont 2 instances distinctes
  - Chaque classe peut avoir sa propre implémentation de l'égalité mais généralement 2 objets sont égaux si tout ou partie de leurs états sont égaux

```
String chaine1 = new String("test");  
String chaine2 = new String("test");  
boolean isSame = (chaine1 == chaine2);  
System.out.println(isSame);  
boolean isEqual = (chaine1.equals(chaine2));  
System.out.println(isEqual);
```

false

true

- Pour rappel :
  - L'opérateur == vérifie si 2 objets sont identiques : comparaison des références mémoire (sont-ils le même objet ?)

# Quelques méthodes particulières...

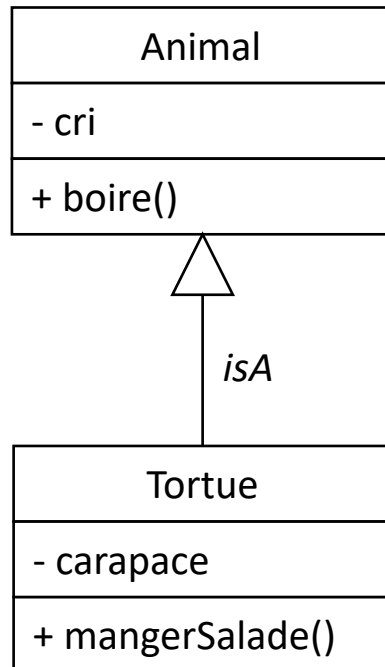
- hashCode()
  - Retourne la valeur de hachage calculée sur l'instance d'un objet
  - Cette valeur de hachage est essentiellement utilisée par les collections de type HashXXX (Hashtable, HashMap, HashSet...) afin d'améliorer leur performance

```
@Override
public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + ((codePIN == null) ? 0 : codePIN.hashCode());
    return result;
}
```

Java 1.7 et supérieur :

```
@Override
public int hashCode() {
    return Objects.hash(codePIN);
}
```

- L'héritage est un **lien** entre des classes. Si une classe est créée à partir d'une classe déjà existante, elle **hérite** de ses attributs et de ses méthodes.



```

public class Animal {

    private String cri;

    public void boire() {
        ...
    }
    ...
}
  
```

```

public class Tortue extends Animal {

    private Boolean carapace;

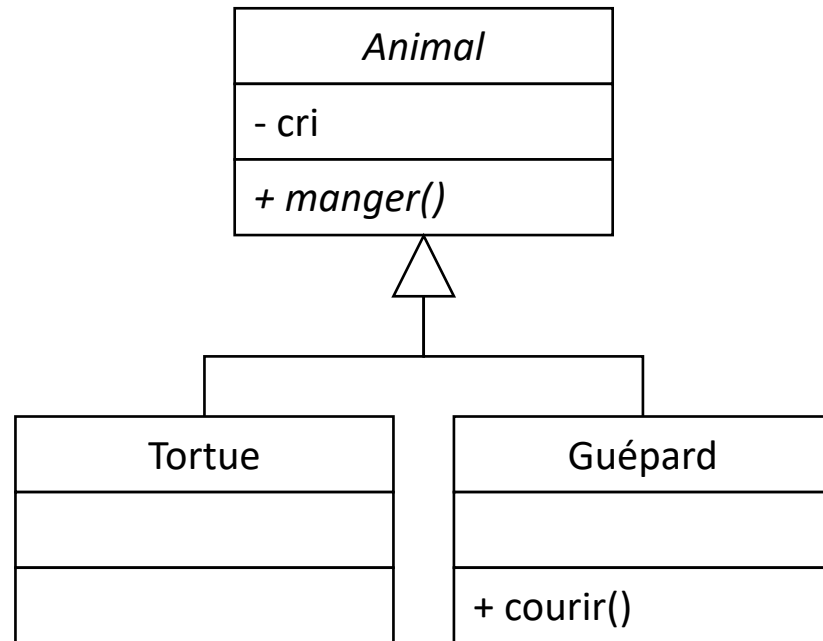
    @Override
    public void boire() {
        // TODO Auto-generated method stub
        super.boire();
    }
    public void mangerSalade() {
        ...
    }
}
  
```

# Les annotations et en particulier @Override

- Types spéciaux en Java qui commencent par @
- Permettent d'ajouter une information à :
  - Une classe, un attribut, une méthode, un paramètre ou une variable
  - Au moment de la compilation, du chargement de la classe dans la JVM ou lors de l'exécution du code
- En Java « classique » peu d'annotations mais en Java EE de nombreuses annotations
- @Override utilisé pour le polymorphisme
  - S'ajoute au début de la signature d'une méthode pour préciser que cette méthode est une redéfinition d'une méthode héritée.
  - Permet au compilateur de vérifier que la signature de la méthode correspond bien à une méthode d'une classe parente. Dans le cas contraire, la compilation échoue.

## Classe abstraite – 1/2

- Une **classe abstraite** est quasiment identique à une classe normale sauf qu'elle **n'est pas instanciable**. Elle peut contenir **des méthodes abstraites**.



```

public abstract class Animal {

    private String cri;

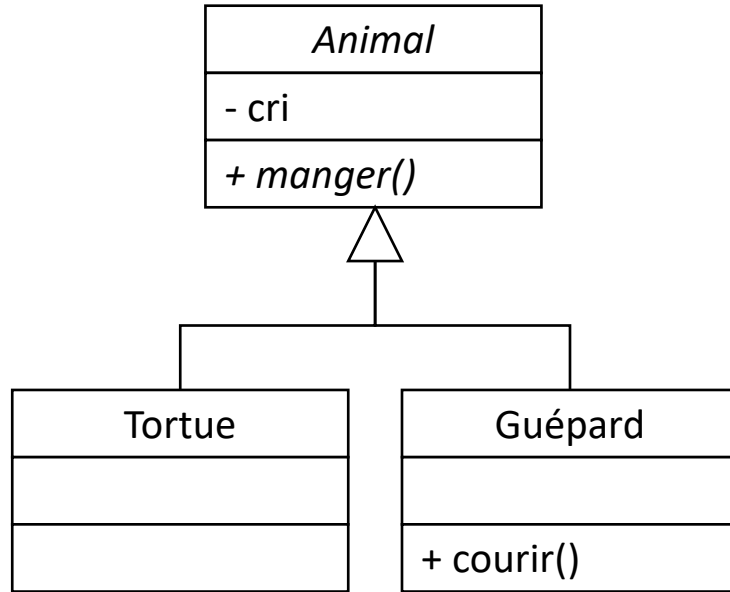
    protected Animal(String cri) {
        this.cri = cri;
    }

    public abstract void manger();

    public String getCri() {
        return cri;
    }
}
  
```

- Le corps de la méthode abstraite est spécifié dans toutes les classes filles et il est **spécifique à chaque classe fille**.

## Classe abstraite – 2/2



```

public class Tortue extends Animal {

    public void manger() {
        System.out.println("Salade");
    }
}
  
```

```

public class Guepard extends Animal
{

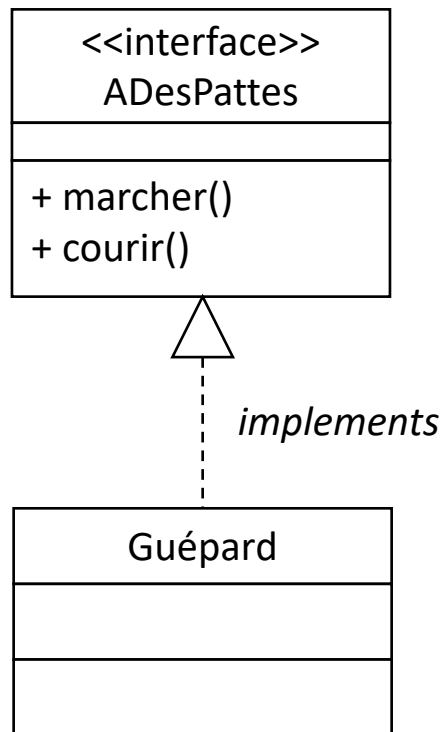
    public void manger() {
        System.out.println("Viande");
    }
    public void courir() {
        System.out.println("Courir");
    }
}
  
```

```

Tortue tortue = new Tortue("wigu");
tortue.manger();
Guepard guepard = new Guepard("rrrr");
guepard.manger();
guepard.courir();
Animal animal = new Animal("cri");
Tortue tortue2 = new Animal("wiwi");
  
```

# Les interfaces

- Les interfaces permettent de définir des méthodes devant être supportées par un objet **sans avoir à fournir l'implémentation** de ces méthodes



```

public interface ADesPattes {
    public void marcher();
    public void courir();
}

```

```

public class Guepard implements ADesPattes {
    public void marcher() {
        // Code
    }
    public void courir() {
        // Code
    }
}

```

```

ADesPattes animal1 = new Guepard();
animal1.marcher();
Guepard guepard = new Guepard();
guepard.marcher();
ADesPattes animal2 = new ADesPattes();

```

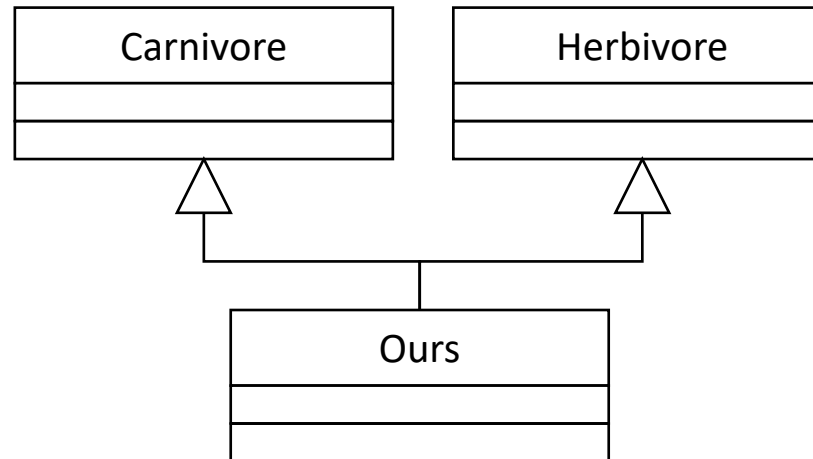
# Classe abstraite ou interface ?

- Pourquoi choisir une classe abstraite plutôt qu'une interface (et inversement) ?

```
public interface ADesPattes {  
    public void marcher();  
    public void courir();  
}
```

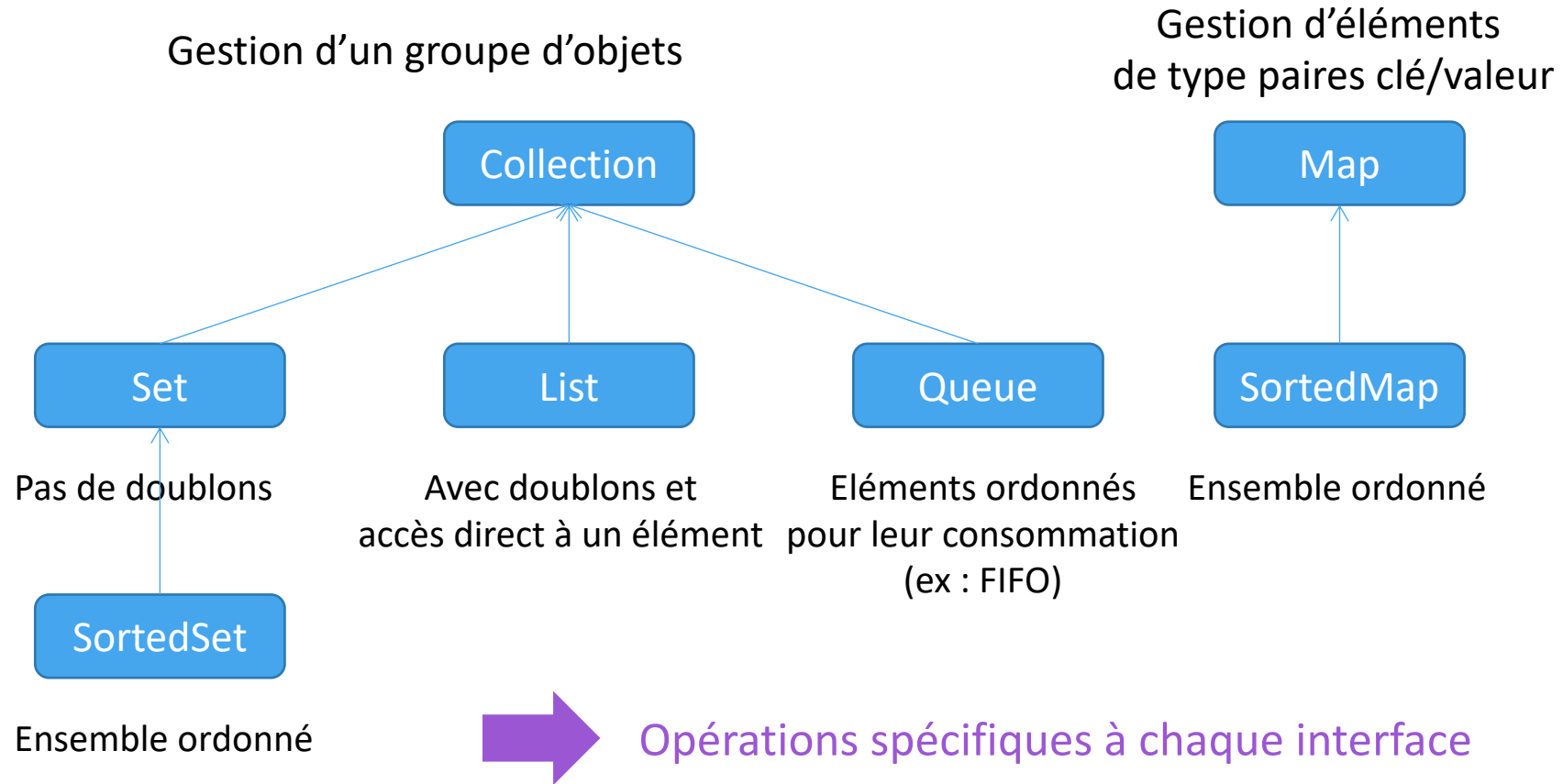
```
public abstract class ADesPattes {  
    public abstract void marcher();  
    public abstract void courir();  
}
```

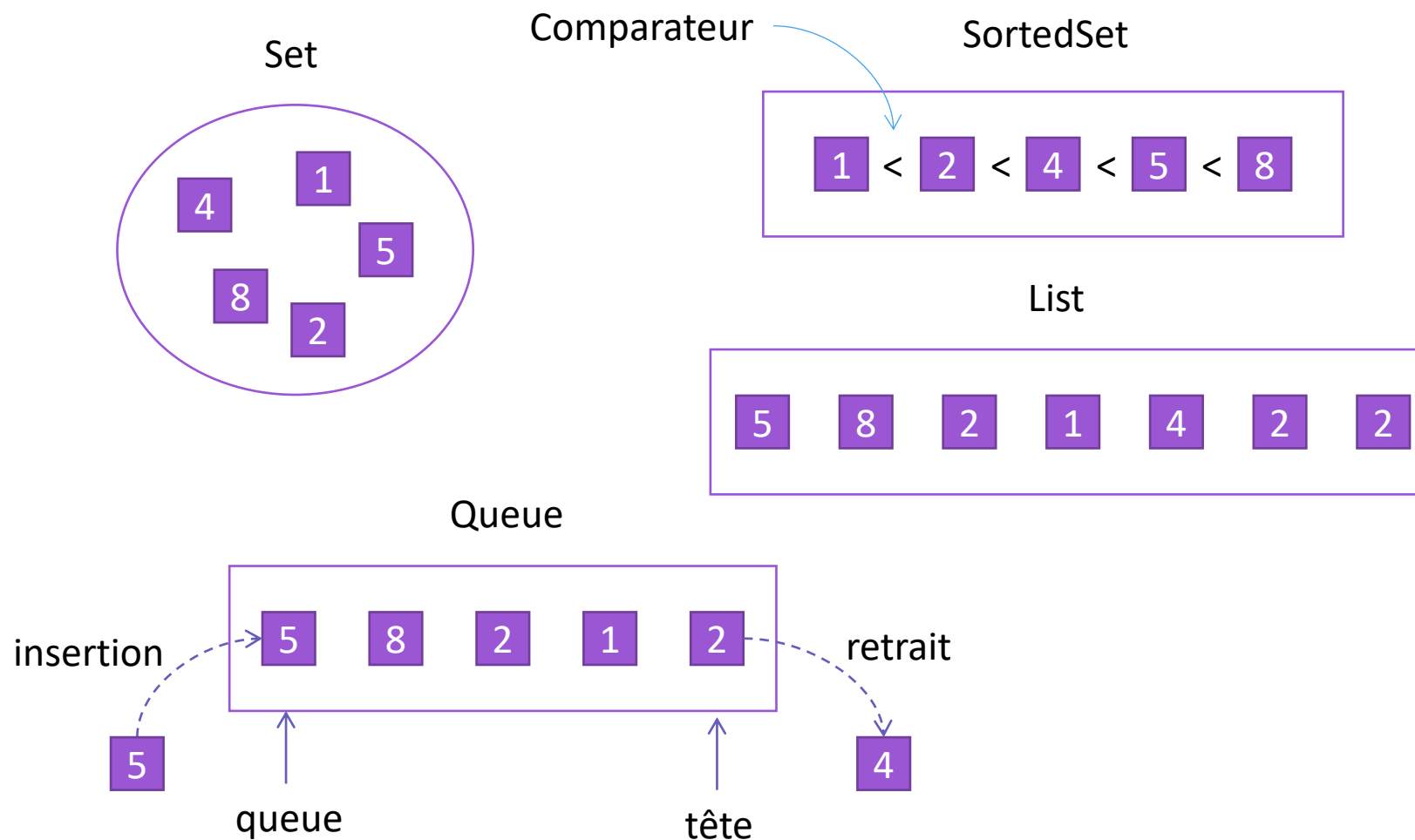
- Est-il possible de définir des attributs dans une interface ?
- Comment faire un héritage multiple en Java ?



# Les collections en Java – 1/4

- Principales interfaces disponibles (java.util.XXX) :



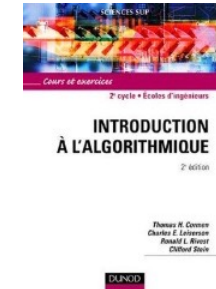
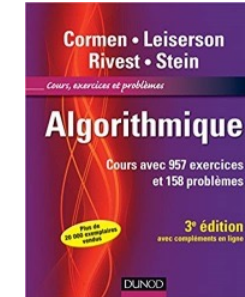


- Classes implémentant les interfaces :

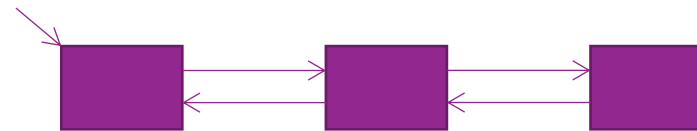
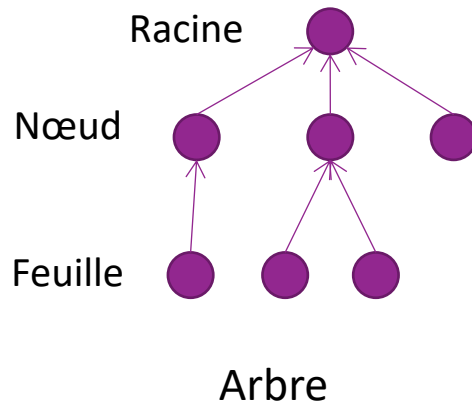
|            |       | Utilisation générale                      | Gestion des accès concurrents                                                                                                                        |
|------------|-------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interfaces | List  | ArrayList<br>LinkedList                   | Vector<br>Stack<br>CopyOnWriteArrayList                                                                                                              |
|            | Set   | HashSet<br>TreeSet<br>LinkedHashSet       | CopyOnWriteArrayList<br>ConcurrentSkipListSet                                                                                                        |
|            | Map   | HashMap<br>TreeMap<br>LinkedHashMap       | Hashtable<br>ConcurrentHashMap<br>ConcurrentSkipListMap                                                                                              |
|            | Queue | LinkedList<br>ArrayDeque<br>PriorityQueue | ConcurrentLinkedQueue<br>LinkedBlockingQueue<br>ArrayBlockingQueue<br>PriorityBlockingQueue<br>DelayQueue<br>SynchronousQueue<br>LinkedBlockingDeque |

- Comment choisir une structure de données ?

T. Cormen *et al.*, Introduction à l'algorithmique, 3<sup>ème</sup> édition, Dunod



➡ Complexité algorithmique des opérations (insertion, recherche, suppression...)



Liste chaînée

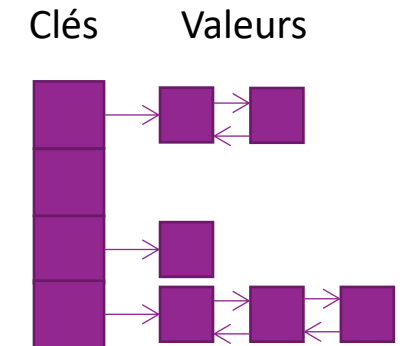
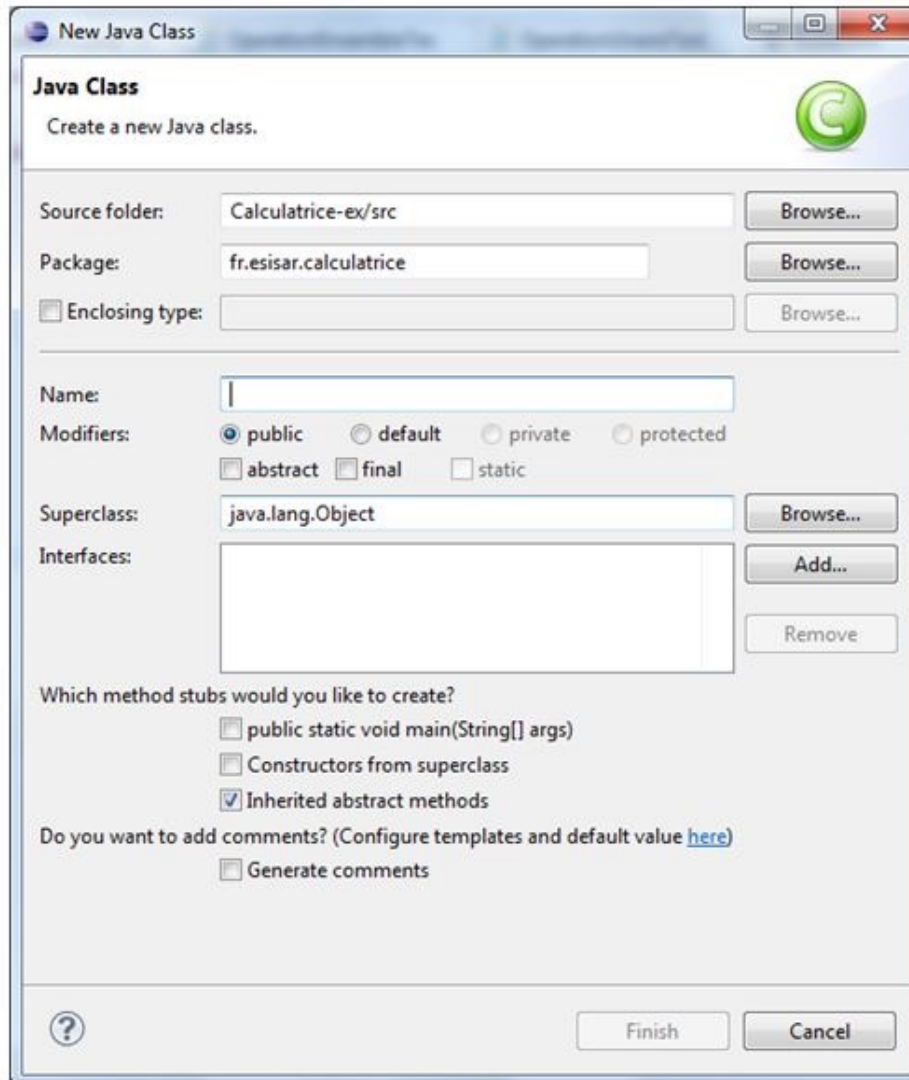


Table de hachage



Répertoire dans lequel va être stocké la classe

Package de la classe

Nom de la classe

Caractéristiques de la classe (visibilité, abstraite...)

Nom de la super-classe pour cette classe

Ensemble des interfaces implantées par la classe

Méthodes dont le squelette sera  
implantée pour la classe

Génération des commentaires

# Eclipse est vraiment votre ami !



|                      |               |                          |
|----------------------|---------------|--------------------------|
| Undo Typing          | Ctrl+Z        | Commentaires             |
| Revert File          |               |                          |
| Save                 | Ctrl+S        |                          |
| Open Declaration     | F3            | Indentation et formatage |
| Open Type Hierarchy  | F4            |                          |
| Open Call Hierarchy  | Ctrl+Alt+H    |                          |
| Show in Breadcrumb   | Alt+Shift+B   | Gestion des imports      |
| Quick Outline        | Ctrl+O        |                          |
| Quick Type Hierarchy | Ctrl+T        |                          |
| Open With            | ▶             | Génération de méthodes   |
| Show In              | Alt+Shift+W ▶ |                          |
| Cut                  | Ctrl+X        |                          |
| Copy                 | Ctrl+C        | Renommage                |
| Copy Qualified Name  |               |                          |
| Paste                | Ctrl+V        |                          |
| Raw Paste            |               |                          |
| Quick Fix            | Ctrl+1        |                          |
| Source               | Alt+Shift+S ▶ |                          |
| Refactor             | Alt+Shift+T ▶ |                          |

|                                          |                             |                          |
|------------------------------------------|-----------------------------|--------------------------|
| Toggle Comment                           | Ctrl+7                      | Commentaires             |
| Remove Block Comment                     | Ctrl+Shift+\<br>Alt+Shift+J |                          |
| Generate Element Comment                 |                             |                          |
| Correct Indentation                      | Ctrl+I                      | Indentation et formatage |
| Format                                   | Ctrl+Shift+F                |                          |
| Format Element                           |                             |                          |
| Add Import                               | Ctrl+Shift+M                | Gestion des imports      |
| Organize Imports                         | Ctrl+Shift+O                |                          |
| Sort Members...                          |                             |                          |
| Clean Up...                              |                             | Génération de méthodes   |
| Override/Implement Methods...            |                             |                          |
| Generate Getters and Setters...          |                             |                          |
| Generate Delegate Methods...             |                             | Renommage                |
| Generate hashCode() and equals()...      |                             |                          |
| Generate toString()...                   |                             |                          |
| Generate Constructor using Fields...     |                             |                          |
| Generate Constructors from Superclass... |                             |                          |
| Externalize Strings...                   |                             |                          |

# Documentation du code

Javadoc

- Outil, conçu initialement par Sun Microsystems et développé aujourd'hui par Oracle, pour créer de la documentation au format HTML

OVERVIEW MODULE PACKAGE **CLASS** USE TREE DEPRECATED INDEX HELP

ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD    DETAIL: FIELD | CONSTR | METHOD

**Module** java.base  
**Package** java.lang  
**Class** Object

java.lang.Object

---

**public class Object**

Class Object is the root of the class hierarchy. Every class has Object as a superclass. All objects, including arrays, implement the methods of this class.

**Since:**  
 1.0

**See Also:**  
 Class

**Constructor Summary**

| Constructors | Description              |
|--------------|--------------------------|
| Object()     | Constructs a new object. |

**Method Summary**

| All Methods       | Instance Methods   | Concrete Methods                                                            | Deprecated Methods |
|-------------------|--------------------|-----------------------------------------------------------------------------|--------------------|
| Modifier and Type | Method             | Description                                                                 |                    |
| protected Object  | clone()            | Creates and returns a copy of this object.                                  |                    |
| boolean           | equals(Object obj) | Indicates whether some other object is "equal to" this one.                 |                    |
| protected void    | finalize()         | <b>Deprecated.</b><br>The finalization mechanism is inherently problematic. |                    |
| Class<?>          | getClass()         | Returns the runtime class of this Object.                                   |                    |
| int               | hashCode()         | Returns a hash code value for the object.                                   |                    |

- La Javadoc s'appuie sur le code source et sur un type de commentaires particuliers
- Les commentaires pour la Javadoc :
  - Commencent par `/**` et finissent `*/`
  - Peuvent contenir un texte libre
  - Peuvent contenir des balises particulières

```
/**  
 * Description  
 *  
 * @tag1  
 * @tag2  
 */
```

# Les balises

| Balises               | Description                                    | Remarques                                                                |
|-----------------------|------------------------------------------------|--------------------------------------------------------------------------|
| @author               | Nom du développeur                             | S'applique à une classe ou une interface                                 |
| @deprecated           | Méthode dépréciée                              |                                                                          |
| @exception ou @throws | Documente une exception levée par une méthode  |                                                                          |
| @param                | Définit un paramètre de méthode                |                                                                          |
| @return               | Documente la valeur de retour                  | Interdit pour les constructeurs et les méthodes avec type de retour void |
| @since                | Précise la version de la JDK                   | S'applique à une classe ou une interface                                 |
| @version              | Donne la version d'une classe ou d'une méthode |                                                                          |

- En ligne de commandes, avec la commande javadoc fournit dans le JDK :

```
javadoc -d doc src/fr/esisar/tp/*.java
```

Répertoire où la Javadoc va être générée  
(des répertoires et des fichiers HTML)

Les fichiers sources

# Tests

JUnit

- Un test est une expérience d'exécution pour mettre en évidence des fautes en observant des erreurs
- Types de tests :
  - **Tests unitaires** : tester les composants isolément
    - Cible : méthodes et classes
  - **Tests d'intégration** : tester un ensemble de composants assemblés
    - Cible : composants et système global
  - **Tests système** : tester le logiciel dans son futur environnement d'exploitation
    - Cible : système global dans un environnement iso-production
  - **Tests d'acceptation (recette)** : tester dans les conditions définies par le futur utilisateur
    - Cible : système global

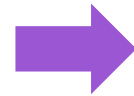
# Caractéristiques d'un test

- Un test doit :
  - Etre reproductible
  - Indépendant de l'environnement
    - Ne dépend pas de l'historique, du contenu d'une base de données...
  - Sans impact pour son environnement
    - Ne modifie pas les conditions pour les autres tests



On peut passer les tests dans n'importe quel ordre,  
un nombre quelconque de fois !

- Framework open-source de tests unitaires pour le langage Java



## Automatisation du processus de tests

- Intérêt :
  - Facilite l'écriture des programmes de tests unitaires
  - Facilite l'exécution des tests unitaires
  - Facilite l'exploitation des résultats des tests unitaires
- Concept principal :
  - Ecrire une classe de tests contenant des méthodes de tests pour vérifier le bon fonctionnement d'une classe

# Exemple de classe de tests

- Remarques :
  - Habituellement le nom de la classe de test est le nom de la classe testée avec le suffixe « Test »
    - Exemple : Calculatrice => CalculatriceTest
  - Le nom des méthodes est généralement long, il décrit le test
  - La visibilité de la classe et des méthodes est habituellement « package »

```
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

class MonTest {

    @Test
    void simpleTest() {
        System.out.println("simpleTest");
        Assertions.assertTrue(true);
    }
}
```

# Quelques assertions... de `org.junit.jupiter.api.Assertions.*`

- Quelques assertions applicables à tous les types : <https://junit.org/>

| Egalité                        | Nullité                      | Exceptions                  |
|--------------------------------|------------------------------|-----------------------------|
| <code>assertEquals()</code>    | <code>assertNull()</code>    | <code>assertThrows()</code> |
| <code>assertNotEquals()</code> | <code>assertNotNull()</code> |                             |
| <code>assertTrue()</code>      |                              |                             |
| <code>assertFalse()</code>     |                              |                             |

- Exemple :

```

@Test
void verifierNull() {
    Object obj = new Dimension(800, 600);
    Assertions.assertNull(obj);
}
  
```

- Résultat :

```

org.opentest4j.AssertionFailedError:
expected: <null> but was: <java.awt.Dimension[width=800,height=600]>
  
```

```

class MonTest {
    @BeforeAll
    static void initAll() {
        System.out.println("beforeAll");
    }

    @BeforeEach
    void init() {
        System.out.println("beforeEach");
    }

    @AfterEach
    void tearDown() {
        System.out.println("afterEach");
    }

    @AfterAll
    static void tearDownAll() {
        System.out.println("afterAll");
    }
  
```

```

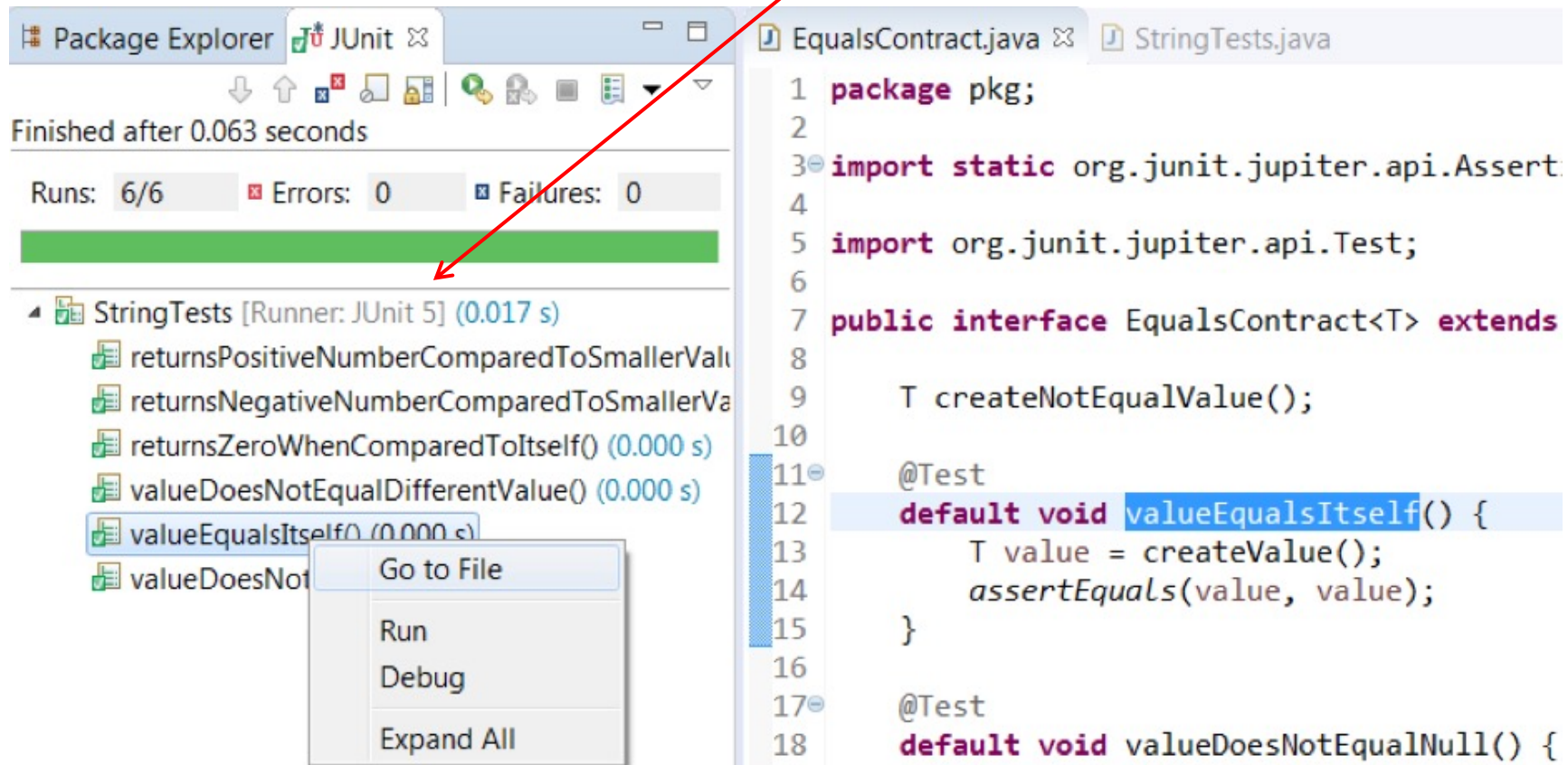
@Test
void simpleTest() {
    System.out.println("simpleTest");
    Assertions.assertTrue(true);
}

@Test
void secondTest() {
    System.out.println("secondTest");
    Assertions.assertTrue(true);
}
  
```

Quelle est la trace d'exécution ?

- Intégré par défaut dans Eclipse

Attention ! Eclipse intègre JUnit 4 et JUnit 5



The screenshot shows the Eclipse IDE interface. On the left, the 'Package Explorer' and 'JUnit' tabs are visible. Below them, a progress bar indicates 'Finished after 0.063 seconds' with 'Runs: 6/6', 'Errors: 0', and 'Failures: 0'. A tree view shows the test results for 'StringTests [Runner: JUnit 5] (0.017 s)', listing several successful tests like 'returnsPositiveNumberComparedToSmallerValue' and 'valueEqualsItself()'. A context menu is open over the 'valueEqualsItself()' test, showing options: 'Go to File', 'Run', 'Debug', and 'Expand All'. On the right, the 'StringTests.java' source file is open, showing imports for JUnit 5 and the implementation of the 'valueEqualsItself()' test method. A red arrow points from the 'Attention ! Eclipse intègre JUnit 4 et JUnit 5' text to the 'JUnit' tab in the Package Explorer.

```

1 package pkg;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5 import org.junit.jupiter.api.Test;
6
7 public interface EqualsContract<T> extends
8     T createNotEqualValue();
9
10
11 @Test
12 default void valueEqualsItself() {
13     T value = createValue();
14     assertEquals(value, value);
15 }
16
17 @Test
18 default void valueDoesNotEqualNull() {
  
```

# Dépendances (bibliothèques) requises pour JUnit 5

- org.junit.jupiter

|                      |                                                 |
|----------------------|-------------------------------------------------|
| junit-jupiter-api    | API pour l'écriture de tests avec JUnit Jupiter |
| junit-jupiter-engine | Implantation du moteur d'exécution              |
| junit-jupiter-params | Support des tests paramétrés                    |

- org.junit.platform

|                                                                             |                                                                                                                           |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| junit-platform-commons                                                      | Utilitaires à usage interne de JUnit                                                                                      |
| junit-platform-console<br>junit-platform-console-standalone                 | Support pour la découverte et l'exécution des tests dans la console                                                       |
| junit-platform-engine<br>junit-platform-launcher<br>(junit-platform-runner) | API publique pour les moteurs d'exécution des tests<br>Runner pour exécuter des tests JUnit5 dans un environnement JUnit4 |
| junit-platform-suite-api                                                    | Support pour l'exécution de suite de tests                                                                                |